C Programming

Assignment 2 (CO2) - Report

Operators, Decision-Making Statements and Looping Constructs

Questions 1 to 4: Precedence and Associativity, Short-Circuit Evaluation, Bitwise Operators, else-if Ladder vs switch

Student name	P.SAI SUSHANTH REDDY
Register number	1925260171
Course / Faculty	
GitHub repository	https://github.com/nagagowthamjanjanam-code/CSA02-1965260183-Assignment-2
Compiler used for measurements	GCC 13.3.0 (Ubuntu 24.04). Compile command: `gcc -Wall -Wextra -o qN qN.c`
Screenshots	Student's own execution screenshots from an online C compiler (Figures 1 to 5)

Fill in the name, register number, faculty and the GitHub link before submission. The assignment states that a report without a working GitHub link is not evaluated.

Contents and marks

Section	Topic	Marks
Q1	Precedence, associativity and the discount bug (K4)	6
Q2	When does short-circuit evaluation matter? (K4)	4
Q3	Device-status register with bitwise operators, Complete the Code (K4)	6
Q4	else-if ladder or switch? Complete the Code (K4)	6
Appendix	Repository layout, build commands and reproducibility	-

Questions 5 to 7 are not part of this document.

Method used in every section

Each question follows the order required by the assignment: **Predict, Design, Implement, Measure, Analyse, Evaluate**. Predictions were worked out by hand from the C rules before any program was run. Raw counts (function calls, comparisons, loop iterations) are the primary evidence; the screenshots only confirm the final output. Counts are deterministic and can be reproduced exactly; the few timing figures depend on the machine and are used only as supporting observations.

Q1 - Precedence, Associativity and the Discount Bug

Jan Aushadhi Pharmacy billing. 6 marks, Bloom K4 (Analyse).

1.1 Problem analysis

Intended rule: the amount payable is price x quantity, **less 15% senior-citizen discount**, **plus 5% GST**, and then Rs 50 is deducted when the total is above Rs 1000. The assignment does not say which amount GST is charged on. I took the usual billing reading, **GST on the discounted amount**, and the corrected program (Figure 1) uses the same reading.

The developer's code:

```
total = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
net = total > 1000 ? total - 50 : total;   /* Rs 50 off above Rs 1000 */
count = count++ + 1;
```

Precedence and associativity rules that apply

Level (high to low)	Operators used here	Associativity
1 (highest)	( )	-
2	* / (same level)	left to right
3	+ - (same level)	left to right
4	>	left to right
5	?:	right to left
6 (lowest)	=	right to left

Precedence decides only *grouping*. It does not fix the order in which the three terms of the sum are evaluated; the compiler may evaluate them in any order, which does not matter here because the terms have no side effects.

1.2 Predictions before execution

Variables were assumed to be **int**, so every / truncates. Three bills were chosen: A (price 200, qty 6, senior), B (price 99, qty 3, senior) and C (price 50, qty 7, not senior).

Operator-by-operator evaluation of the original expression, Bill A (200, 6, senior = 1)

Step	Operation (grouping given by precedence and left-to-right associativity)	Result
1	price * qty (first * of term 1)	1200
2	(price * qty) * 15 (left-to-right, so multiplication before division)	18000
3	18000 / 100	180
4	180 * senior	180
5	(price * qty) * 5, then / 100 (term 3)	6000, then 60
6	1200 - 180 (left-to-right - and +)	1020
7	1020 + 60	1080
8	total = 1080 (assignment is last, lowest precedence)	1080
9	total > 1000, then ?: selects total - 50	1030

Associativity trap: because * and / are left-associative, `price*qty*15/100` multiplies by 15 *before* dividing by 100, which keeps the precision. Had the grouping been `price*qty*(15/100)`, then 15/100 = 0 in integer arithmetic and the discount would vanish (measured: 0).

Bill	Hand prediction, original expression (int)	Hand prediction, double variables	Intended (GST on discounted amount)
A (200, 6, 1)	total 1080, net 1030	1080.00 / 1030.00	gross 1200, discount 180, GST 51, total 1071, net 1021
B (99, 3, 1)	297; 4455/100 = 44; 1485/100 = 14; 297 - 44 + 14 = 267; net 267	297 - 44.55 + 14.85 = 267.30	discount 44.55, GST 12.6225, total 265.07
C (50, 7, 0)	350 - 0 + 1750/100 (= 17) = 367; net 367	350 + 17.5 = 367.50	GST 17.50, total 367.50

1.3 Implementation and observed results

Program q1_compare.c evaluates the original expression with int variables, with double variables, and the corrected calculation, for the three bills.

```
#include <stdio.h>
/* Q1: buggy single-expression billing vs corrected, three bills */
typedef struct { int price, qty, senior; } Bill;

int main(void)
{
    Bill b[3] = { {200, 6, 1}, {99, 3, 1}, {50, 7, 0} };
    printf("%-12s %-10s %-10s %-10s %-10s %-10s %-10s\n",
           "Bill(p,q,s)", "int total", "int net", "dbl total", "dbl net", "right tot", "right net");
    for (int i = 0; i < 3; i++) {
        int price = b[i].price, qty = b[i].qty, senior = b[i].senior;

        /* original expression, all-int variables (integer division truncates) */
        int itotal = price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100;
        int inet   = itotal > 1000 ? itotal - 50 : itotal;

        /* original expression, double variables (no truncation, precedence bug stays) */
        double dp = price, dq = qty, ds = senior;
        double dtotal = dp * dq - dp * dq * 15 / 100 * ds + dp * dq * 5 / 100;
        double dnet   = dtotal > 1000 ? dtotal - 50 : dtotal;

        /* corrected: GST on the discounted amount */
        float gross = (float)price * qty;
        float discount = senior ? gross * 15 / 100 : 0;
        float tax = (gross - discount) * 5 / 100;
        float total = gross - discount + tax;
        float net = (total > 1000) ? total - 50 : total;

        printf("(%d,%d,%d)%*s %-10d %-10d %-10.2f %-10.2f %-10.2f %-10.2f\n",
               price, qty, senior, 4, "", itotal, inet, dtotal, dnet, total, net);
    }
    return 0;
}
```

```
Bill(p,q,s)  int total  int net    dbl total  dbl net    right tot  right net
(200,6,1)      1080       1030       1080.00    1030.00    1071.00    1021.00
(99,3,1)      267        267        267.30     267.30     265.07     265.07
(50,7,0)      367        367        367.50     367.50     367.50     367.50
```

Bill	Predicted total/net (int)	Observed int total/net	Observed double total	Intended total/net	Error of original	Verdict
A	1080 / 1030	1080 / 1030	1080.00	1071.00 / 1021.00	+9.00 (Rs 9 overcharged)	WRONG
B	267 / 267	267 / 267	267.30	265.07 / 265.07	+1.93 (int)	WRONG
C	367 / 367	367 / 367	367.50	367.50 / 367.50	-0.50 (int) / 0.00 (double)	wrong in int only

All predictions matched the observed output exactly. The step-by-step trace program q1_steps.c reproduced the Bill A table above:

```
T1=price*qty=1200
T2: price*qty=1200, *15=18000, /100=180, *senior=180
T3: price*qty=1200, *5=6000, /100=60
T1 - T2 = 1020, + T3 = 1080
price*qty*(15/100)*senior = 0  (15/100 is 0 in integer arithmetic)
```

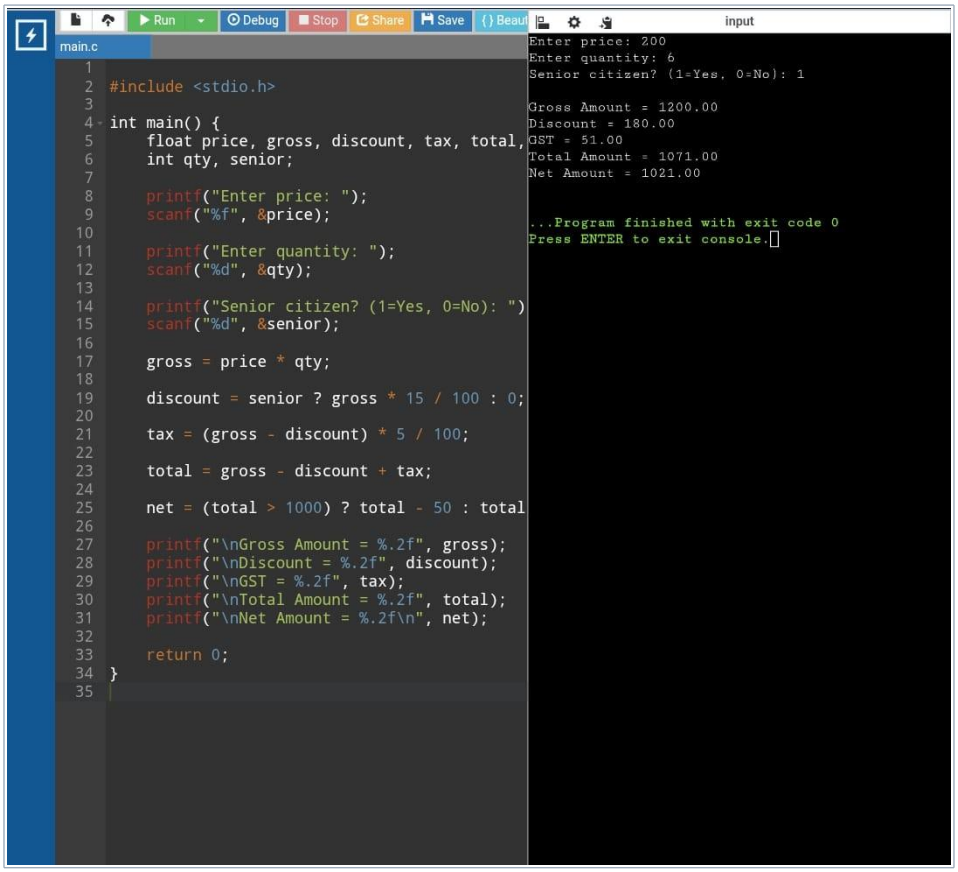


Figure 1 - Student's execution of the corrected program (price 200, qty 6, senior = 1): gross 1200, discount 180, GST 51, total 1071, net 1021. This matches the hand-calculated intended values.

Expressions that differ from what the pharmacist intended

- **GST base (logic error).** The original charges 5% on `price*qty` (the gross amount) instead of the discounted amount. For a senior citizen this overcharges by 5% of the discount: Rs 9 on Bill A (5% of 180).
- **Integer division.** Each `/ 100` truncates before the terms are combined: 4455/100 gives 44 (true 44.55), 1485/100 gives 14 (true 14.85), 1750/100 gives 17 (true 17.50). Money amounts are lost in paise, up to Rs 1 per term.
- **Net amount.** The Rs 50 rule is applied to the wrong total, so the error is carried into `net` (Bill A: 1030 instead of 1021). A bill near Rs 1000 could even be put on the wrong side of the threshold.
- **Third statement.** `count = count++ + 1;` has undefined behaviour (next section).

1.4 Inquiry: why is count = count++ + 1 undefined behaviour?

In this statement the object *count* is modified twice, once by the post-increment and once by the assignment, and its value is also read for the addition. C11 (6.5p2) says that if a side effect on a scalar object is **unsequenced** relative to another side effect on the same object, or to a value computation using that object, the behaviour is undefined. The assignment is sequenced after the value computations of its operands, but nothing orders the store done by ++ against the store done by =. A compiler may therefore produce count+1, produce the old value, or anything else.

What GCC does with -Wall. It compiles the file but prints a warning; it does not stop:

```
q1_count.c: In function 'main':
q1_count.c:6:11: warning: operation on 'count' may be undefined [-Wsequence-point]
    6 |     count = count++ + 1;
      |     ~~~~~~^~~~~~~~~~~~
```

With count = 5, the program printed **count = 6** at -O0, -O2 and -O3 on GCC 13.3. That agreement is only luck of this compiler version; the standard does not guarantee it. The intent is simply to add 1, so the correct statements are `count++;` or `count = count + 1;`.

1.5 Design decision: the corrected calculation

```
gross    = price * qty;
discount = senior ? gross * 15 / 100 : 0;      /* multiply first, divide last */
tax      = (gross - discount) * 5 / 100;       /* the one place that needs brackets */
```

```
total    = gross - discount + tax;
net      = (total > 1000) ? total - 50 : total;
```

Decision: use both intermediate variables and brackets.

- Intermediate variables (gross, discount, tax) give each business rule its own line. Each value can be printed (as Figure 1 does) and checked against the hand calculation, so the GST-base bug becomes visible.
- Brackets are used in exactly one place, `(gross - discount)`, where the intended grouping differs from what precedence would give. Brackets everywhere would clutter the code; none at all caused the bug.
- The order `gross * 15 / 100` is kept: multiplying before dividing preserves precision.
- Money should not use int division. The submitted program uses float; double (or integer paise) is safer for real billing because float keeps only about 7 significant digits.

1.6 Alternative concept and complexity analysis

Alternative: replace the single long expression by several statements with intermediate variables, which is exactly the corrected program.

Aspect	Single expression (original)	Intermediate variables (corrected)
Arithmetic operations	10 (price*qty is computed three times; the discount term has 4 operators, the GST term 3, the combination 2)	8 arithmetic operations plus one conditional (gross: 1, discount: 2, tax: 3, total: 2)
Time	O(1), 10 operations	O(1), 8 operations. Saves repeated price*qty. With -O2 the compiler usually removes the repetition anyway, so the real saving is small.
Extra space	none declared	3 temporaries (gross, discount, tax) = 12 bytes as float, O(1); normally kept in registers
Correctness	GST base wrong; hard to see	Each step can be tested separately; bug removed
Readability	Needs a precedence table to read	One rule per line, self-documenting

Conclusion: time is O(1) in both versions and the extra space is negligible. Intermediate variables win on correctness and readability, and slightly on operation count.

Pseudocode (corrected)

```
READ price, qty, senior
gross    <- price * qty
IF senior THEN discount <- gross * 15 / 100 ELSE discount <- 0
tax      <- (gross - discount) * 5 / 100
total    <- gross - discount + tax
IF total > 1000 THEN net <- total - 50 ELSE net <- total
PRINT gross, discount, tax, total, net
```

Q2 - When Does Short-Circuit Evaluation Matter?

Railway senior-citizen concession. 4 marks, Bloom K4 (Analyse).

2.1 Problem analysis and experiment design

Rule used in the experiments: **concession = (senior AND eligible class) OR (disability-card holder AND card verifies)**. The function `verify_card()` stands for the slow server call; every call increments a counter. Passenger data are generated by a fixed pseudo-random generator, so every version sees exactly the same 1000 passengers.

Generated data (target 25% senior, 40% eligible class, 8% card holders). Actual counts: senior = 248, eligible = 405, card holders = 89, senior AND eligible = 99.

Figure 2 is the student's interactive single-passenger program using the rule `(senior && eligible) || disability || verify_card()`. With inputs 1, 1, 0 it printed "Concession Granted" and **Verification calls = 0**: the left side of the `||` was already true, so the slow function was never called.

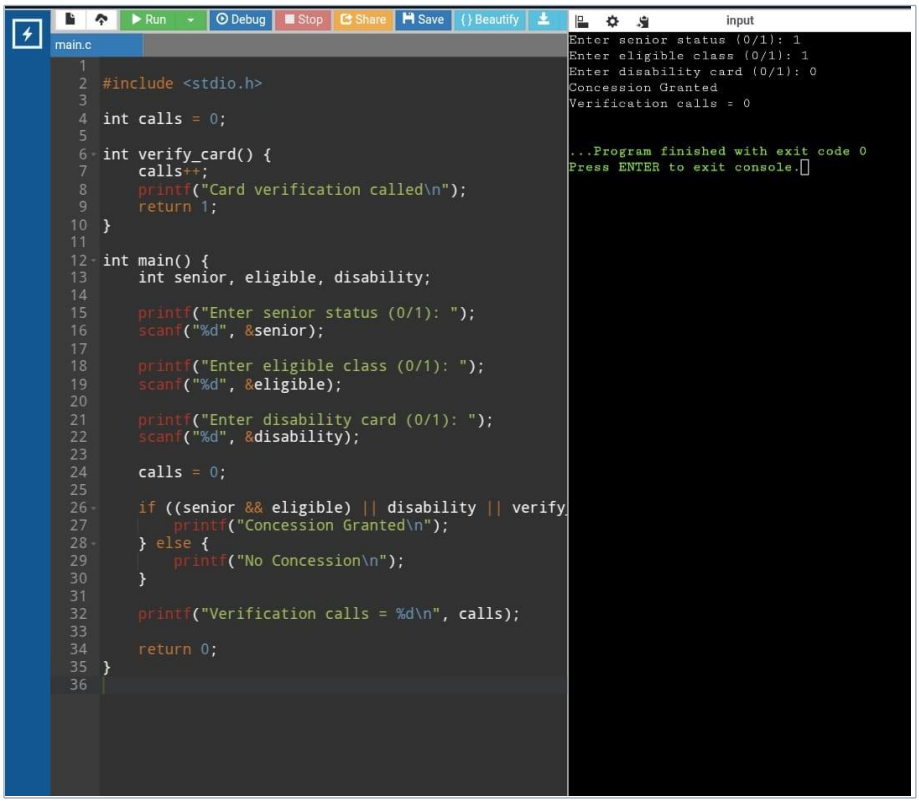


Figure 2 - Student's execution: the slow function is skipped (calls = 0) because (senior && eligible) is already true.

2.2 Experiment 1: when is the second operand not evaluated?

Each side-effect assignment below shows whether the right operand ran (a flag changes to 1 only if it was evaluated).

```
0 && (a=1): result=0 a=0
1 && (b=1): result=1 b=1
1 || (c=1): result=1 c=0
0 || (d=1): result=1 d=1
0 &  (e=1): result=0 e=1
1 |  (f=1): result=1 f=1
2 & 1 = 0    but    2 && 1 = 1
```

Expression	Right operand evaluated?	Reason
0 && (a = 1)	No (a stays 0)	Left is false, so the && result is already 0
1 && (b = 1)	Yes	Left is true, the right operand decides the result
1 || (c = 1)	No (c stays 0)	Left is true, so the || result is already 1
0 || (d = 1)	Yes	Left is false, the right operand decides
0 & (e = 1) and 1 | (f = 1)	Always (e = f = 1)	& and | are bitwise operators: no short-circuit
2 & 1 versus 2 && 1	-	2 & 1 = 0 (bits 10 and 01 share nothing) but 2 && 1 = 1. On conditions that are not exactly 0 or 1, & gives wrong answers.

Difference between & and &&: && is a logical operator that guarantees left-to-right order, stops as soon as the result is known, and treats any non-zero value as true. & is a bitwise operator on the bits of both operands; it always evaluates both and gives no ordering guarantee.

2.3 Experiment 2: order of conditions and the number of slow calls (1000 passengers)

```
V1  (S && E) || (C && verify())     cheap check first, card guard before the slow call
V2  (C && verify()) || (S && E)     card guard first
V3  (verify() && C) || (S && E)     slow call written first
V4  (S & E) | (C & verify())        non-short-circuit operators
V5  nested if (same logic as V1)
V6  (E && S) || (C && verify())     cheap conditions swapped
```

```
V1 (S&&E)||(C&&verify)          slow calls=  78  granted=177  audit entries=  78  senior/elig evals=1000/248 (sum 1248)
V2 (C&&verify)||(S&&E)          slow calls=  89  granted=177  audit entries=  89  senior/elig evals=911/227 (sum 1138)
V3 (verify&&C)||(S&&E)          slow calls=1000  granted=177  audit entries=1000  senior/elig evals=911/227 (sum 1138)
V4 (S&E)|(C&verify) bitwise     slow calls=1000  granted=177  audit entries=1000  senior/elig evals=1000/1000 (sum 2000)
V5 nested if (same as V1)       slow calls=  78  granted=177  audit entries=  78  senior/elig evals=1000/248 (sum 1248)
V6 (E&&S)||(C&&verify)          slow calls=  78  granted=177  audit entries=  78  senior/elig evals=405/1000 (sum 1405)
```

Version	Slow calls	Granted	Cheap evaluations (senior + eligible)	Weighted cost (slow call = 1000 cheap)	Relative to V1
V1	78	177	1248	79,248	1.00
V2	89	177	1138	90,138	1.14
V3	1000	177	1138	1,001,138	12.63
V4	1000	177	2000	1,002,000	12.64
V5	78	177	1248	79,248	1.00
V6	78	177	1405	79,405	1.00

The weight of 1000 cheap evaluations per slow call is an assumption made for illustration; the slow-call counts themselves are measured.

Observations

- All six versions grant the same 177 concessions, so they are logically equivalent. Only the cost differs.
- **V3 and V4 call the slow function 1000 times** (once per passenger): V3 because verify() is written first, V4 because & and | evaluate both sides.
- **V2 calls it 89 times**, exactly the number of card holders, because C guards the slow call.
- **V1 calls it 78 times**: the 89 card holders minus the 11 who already qualified through (senior AND eligible), which is tested first and skips the right side of the ||.
- **V5 (nested if) equals V1 (78 = 78)**: short-circuit operators are just a compact form of the same branching.
- **V1 versus V6:** swapping the cheap conditions does not change the slow calls (78), but cheap evaluations rise from 1248 to 1405. Senior is true for 24.8% of passengers and eligible for 40.5%, so testing senior first stops earlier more often.

2.4 Experiment 3: effect of how many passengers hold a card

```
card%   V1      V2      V3      V4
1       8       8       1000    1000
8       78      89      1000    1000
25      229     251     1000    1000
50      474     517     1000    1000
90      814     893     1000    1000
```

V3 and V4 stay at 1000 whatever the data. V1 and V2 grow with the proportion of card holders and never exceed it. Approximate model: V1 makes n x p_{card} x (1 - $p_{S \text{ and } E}$) calls, V2 makes n x p_{card}, V3 and V4 make n. For 8% card holders: 1000 x 0.089 x 0.901 = 80 (measured 78) and 89 (measured 89).

2.5 When should the cheapest or most-likely-false condition come first?

- **For A && B:** put first the condition that is cheap and likely to be false, because then B is skipped most often. Expected cost = cost(A) + P(A true) x cost(B). Here the card check (false for 91% of passengers, and very cheap) must guard the slow call.
- **For A || B:** put first the condition that is cheap and likely to be *true*, because then B is skipped. This is why (senior && eligible) before the card branch saves 11 slow calls.
- **Order is irrelevant** when both operands are side-effect-free, cost the same and have the same probability, or when both must be evaluated anyway (& and |). It is **never** irrelevant when the first operand guards the second, as in `p != NULL && p->x`.

2.6 Changed condition: the verification function now writes an audit log

In the experiment every call to verify_card() also appends the passenger to an audit log. The audit entries equal the slow calls: V1 = 78, V2 = 89, V3 and V4 = 1000. Re-evaluation:

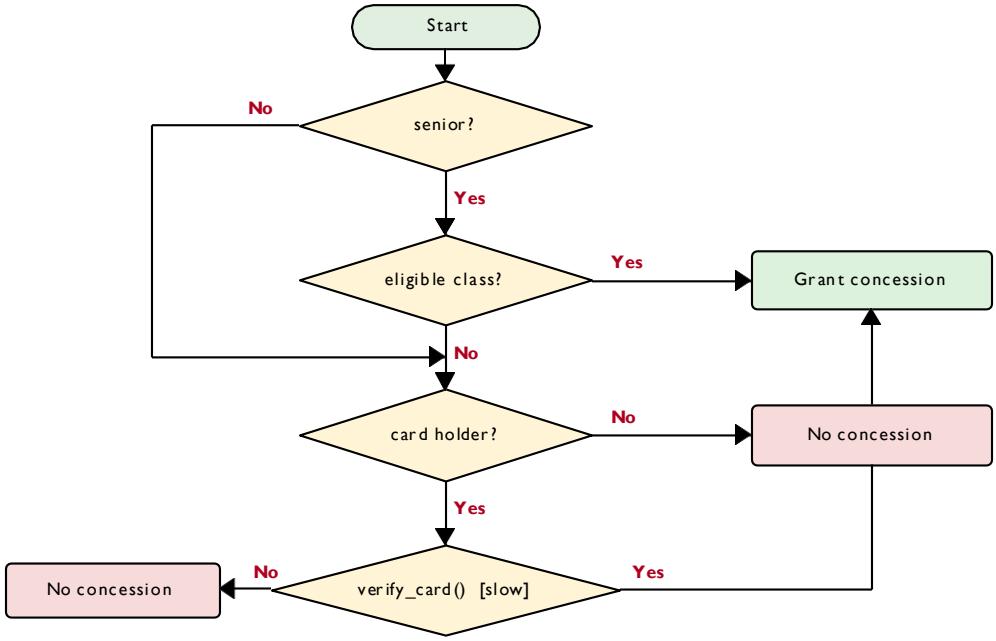
- **V3 and V4 become clearly unacceptable:** they log 911 passengers who hold no card. That wastes storage and one log write per call, and records people who were never verified.
- **V1 versus V2 now depends on the audit rule.** If the log must contain every card holder who presented a card, V1 is wrong because it silently skips the 11 card holders already granted by (senior AND eligible); the log would hold 78 instead of 89 entries. **Recommended order for that policy: V2 (card guard first)**, at the cost of 11 more slow calls (1.1% of passengers).
- If the log only needs the verifications that were actually required for a decision, V1 remains best (fewest calls).
- General rule learned: **do not hide a required side effect in the right operand of && or ||**, because it runs only conditionally. When the log entry is mandatory, call the function explicitly, e.g. `ok = has_card() ? verify_card() : 0;`, then combine the results.

2.7 Alternative concept: nested if, or the non-short-circuit operators

Version	Slow calls for n passengers	Measured (n = 1000)	Time class
&& / || with guard first (V1, V2)	between 0 and n; about n x p_{card} x (1 - $p_{S \text{ and } E}$) for V1, n x p_{card} for V2	78 and 89	O(n) worst case, small constant
Nested if (V5)	same as V1 (the same branches are taken)	78	O(n) worst case, same as V1
Slow call written first, or & and | (V3, V4)	exactly n	1000	Theta(n), always the maximum

Space is O(1) for all versions (only counters), except the audit log, which grows with the number of calls. Nested if gives no speed advantage over a well-ordered && / ||; it is preferable only when each branch needs a different action (for example a different message). The non-short-circuit operators remove the saving completely and should not be used on conditions.

Evaluation flow of V1 / V5 (diagram)



Both No branches of the first two tests join the card check. The slow function is reached only through the card-holder test, which is why V1 and V2 call it so rarely.

Pseudocode

```
FOR each passenger p
    granted <- FALSE
    IF p.senior AND p.eligible THEN granted <- TRUE          (cheap, tested first)
    IF NOT granted AND p.hasCard THEN                          (cheap guard)
        IF verify_card(p) THEN granted <- TRUE                 (slow call, only when needed)
    PRINT granted
```

Source code

```
#include <stdio.h>
#include <stdlib.h>
/* Q2: short-circuit experiments for the railway concession rule.
   Rule: concession = (senior AND eligible class) OR (disability card holder AND card verifies). */
#define N 1000

static unsigned long long seed = 12345;
static int rnd(int mod) { seed = seed * 6364136223846793005ULL + 1442695040888963407ULL; return (int)((seed >> 33) % (unsigned)mod); }

int senior_[N], elig_[N], card_[N];
unsigned long slow_calls, s_calls, e_calls;
int audit_log[N]; int audit_len;
int cur;                                /* index of passenger being processed */

int is_senior(void)   { s_calls++; return senior_[cur]; }
int is_eligible(void) { e_calls++; return elig_[cur]; }
int has_card(void)    { return card_[cur]; }
int verify_card(void) { slow_calls++; audit_log[audit_len++] = cur; return 1; } /* slow server call, card is genuine */

void generate(int pc_senior, int pc_elig, int pc_card)
{
    seed = 12345;
    for (int i = 0; i < N; i++) {
        senior_[i] = rnd(100) < pc_senior;
        elig_[i]   = rnd(100) < pc_elig;
        card_[i]   = rnd(100) < pc_card;
    }
}

typedef int (*Rule)(void);
int v1(void) { return (is_senior() && is_eligible()) || (has_card() && verify_card()); }
int v2(void) { return (has_card() && verify_card()) || (is_senior() && is_eligible()); }
int v3(void) { return (verify_card() && has_card()) || (is_senior() && is_eligible()); }
int v4(void) { return (is_senior() & is_eligible()) | (has_card() & verify_card()); }
int v5(void)
{   /* nested-if equivalent of v1 */
    int g = 0;
    if (is_senior()) { if (is_eligible()) g = 1; }
    if (!g) { if (has_card()) { if (verify_card()) g = 1; } }
    return g;
}
int v6(void) { return (is_eligible() && is_senior()) || (has_card() && verify_card()); } /* cheap order swapped */
```

```
void run(const char *name, Rule r, int show_cheap)
{
    slow_calls = s_calls = e_calls = 0; audit_len = 0;
    int granted = 0;
    for (cur = 0; cur < N; cur++) granted += r();
    printf("%-34s slow calls=%4lu  granted=%3d  audit entries=%4d", name, slow_calls, granted, audit_len);
    if (show_cheap) printf("  senior/elig evals=%lu/%lu (sum %lu)", s_calls, e_calls, s_calls + e_calls);
    printf("\n");
}

int main(void)
{
    printf("== Part 1: 1000 passengers (25%% senior, 40%% eligible class, 8%% card holders) ==\n");
    generate(25, 40, 8);
    int ns = 0, ne = 0, nc = 0, nse = 0;
    for (int i = 0; i < N; i++) { ns += senior_[i]; ne += elig_[i]; nc += card_[i]; nse += senior_[i] && elig_[i]; }
    printf("actual: senior=%d eligible=%d card=%d senior&&eligible=%d\n", ns, ne, nc, nse);
    run("V1 (S&&E)||(C&&verify)",     v1, 1);
    run("V2 (C&&verify)||(S&&E)",     v2, 1);
    run("V3 (verify&&C)||(S&&E)",     v3, 1);
    run("V4 (S&E)|(C&verify) bitwise", v4, 1);
    run("V5 nested if (same as V1)",  v5, 1);
    run("V6 (E&&S)||(C&&verify)",     v6, 1);

    printf("\n== Part 2: card-holder prevalence sweep (slow calls out of 1000) ==\n");
    printf("%-8s %-6s %-6s %-6s %-6s\n", "card%", "V1", "V2", "V3", "V4");
    int sweep[] = {1, 8, 25, 50, 90};
    for (int k = 0; k < 5; k++) {
        generate(25, 40, sweep[k]);
        unsigned long c[4]; Rule rs[4] = {v1, v2, v3, v4};
        for (int j = 0; j < 4; j++) { slow_calls = 0; audit_len = 0; for (cur = 0; cur < N; cur++) rs[j](); c[j] = slow_calls; }
        printf("%-8d %-6lu %-6lu %-6lu %-6lu\n", sweep[k], c[0], c[1], c[2], c[3]);
    }

    printf("\n== Part 3: when is the 2nd operand NOT evaluated? ==\n");
    int a = 0, b = 0, c = 0, d = 0, e = 0, f = 0, g = 0, h = 0;
    int r1 = 0 && (a = 1);   /* && : left false -> right skipped */
    int r2 = 1 && (b = 1);   /* && : left true  -> right evaluated */
    int r3 = 1 || (c = 1);   /* || : left true  -> right skipped */
    int r4 = 0 || (d = 1);   /* || : left false -> right evaluated */
    int r5 = 0 & (e = 1);    /* &  : always evaluates both */
    int r6 = 1 | (f = 1);    /* |  : always evaluates both */
    int r7 = 2 & 1;          /* bitwise: 10 & 01 = 0 although both operands are 'true' */
    int r8 = 2 && 1;         /* logical: 1 */
    (void)g; (void)h;
    printf("0 && (a=1): result=%d a=%d\n", r1, a);
    printf("1 && (b=1): result=%d b=%d\n", r2, b);
    printf("1 || (c=1): result=%d c=%d\n", r3, c);
    printf("0 || (d=1): result=%d d=%d\n", r4, d);
    printf("0 &  (e=1): result=%d e=%d\n", r5, e);
    printf("1 |  (f=1): result=%d f=%d\n", r6, f);
    printf("2 & 1 = %d   but   2 && 1 = %d\n", r7, r8);
    return 0;
}
```

Q3 - Device-Status Register with Bitwise Operators

Smart-home controller. 6 marks, Bloom K4 (Analyse). Includes Complete the Code.

3.1 Problem analysis

One **unsigned char** holds eight device states, bit *n* for device *n* (LIGHT = bit 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4). A mask with only bit *dev* set is `1u << dev`. Each operation uses one property of a bitwise operator:

Operation	Expression	Why it works
Switch on	`reg | (1u << dev)`	x OR 1 = 1 forces the bit to 1; x OR 0 = x leaves the other bits unchanged
Switch off	`reg & ~(1u << dev)`	~mask has a 0 only at bit dev; x AND 0 = 0 clears it, x AND 1 = x keeps the others
Toggle	`reg ^ (1u << dev)`	x XOR 1 flips x; x XOR 0 = x
Test	`(reg & (1u << dev)) != 0`	AND with the mask keeps only bit dev; the result is non-zero exactly when it is 1
Print in binary	`((reg >> bit) & 1u)`	shift bit to position 0, then mask it
Count devices ON	`while (r) { r &= r - 1; n++; }`	r & (r - 1) clears the lowest set bit, so the loop runs once per ON device

3.2 Complete the Code

TODO 1 to 4 were completed using only bitwise operators; the rest of the program is unchanged.

```
#include <stdio.h>
/* Each bit of one byte stores the ON/OFF state of one device. */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
    return reg | (1u << dev);          /* TODO 1: set bit dev */
}
unsigned char switch_off(unsigned char reg, int dev)
{
    return reg & ~(1u << dev);         /* TODO 2: clear bit dev */
}
unsigned char toggle(unsigned char reg, int dev)
{
    return reg ^ (1u << dev);          /* TODO 3: flip bit dev */
}
int is_on(unsigned char reg, int dev)
{
    return (reg & (1u << dev)) != 0;   /* TODO 4: test bit dev */
}
void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}
int main(void)
{
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");
    return 0;
}
```

Prediction: register value after each statement of main()

Statement	Register (binary)	Decimal	Printed by show()
unsigned char reg = 0;	00000000	0	-
reg = switch_on(reg, LIGHT);	00000001	1	-
reg = switch_on(reg, AC);	00000101	5	00000101
reg = toggle(reg, FAN);	00000111	7	-
reg = switch_off(reg, LIGHT);	00000110	6	00000110
is_on(reg, AC) (bit 2)	bit 2 = 1	-	AC is ON
is_on(reg, PUMP) (bit 4)	bit 4 = 0	-	PUMP is OFF

Observed output (program run)	Expected output given in the assignment	Match
00000101 00000110 AC is ON, PUMP is OFF	00000101 00000110 AC is ON, PUMP is OFF	Identical

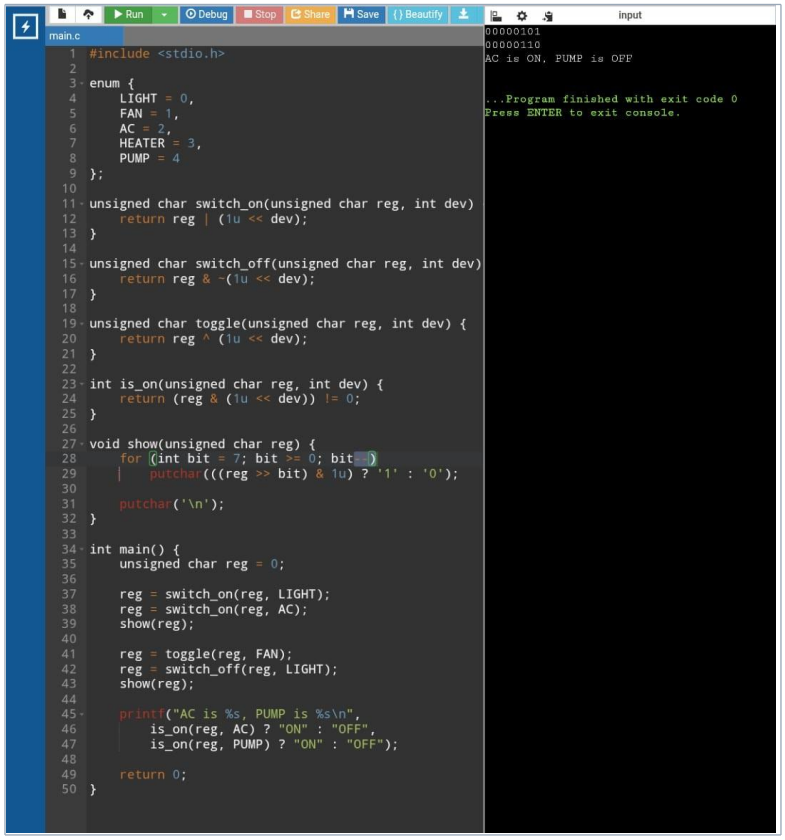


Figure 3 - Student's execution of the completed program: 00000101, 00000110, "AC is ON, PUMP is OFF".

3.3 My own predicted sequence and register trace

A second sequence was designed to include the boundary device (bit 7), an operation that changes nothing (clearing a bit that is already 0) and toggles. Each predicted value was written down first; the program q3_trace.c then compared them.

```
== Part 1: predicted vs observed register after each statement ==
statement              pred.bin  dec  obs.bin   dec  match
switch_on(HEATER)      00001000 8    00001000 8    yes
switch_on(PUMP)        00011000 24   00011000 24   yes
switch_on(FAN)         00011010 26   00011010 26   yes
toggle(HEATER)         00010010 18   00010010 18   yes
toggle(LIGHT)          00010011 19   00010011 19   yes
switch_off(PUMP)       00000011 3    00000011 3    yes
switch_off(AC)         00000011 3    00000011 3    yes
switch_on(device 7)    10000011 131  10000011 131  yes
devices ON in final register = 3 (loop iterations = 3)
```

All eight predictions matched. Note that switch_off(AC) on a bit that is already 0 leaves the register unchanged (idempotent), and that three devices are ON at the end (LIGHT, FAN, device 7). The Kernighan counting loop ran exactly 3 iterations for 3 ON devices.

3.4 Inquiry: precedence and signedness

reg & 1 << dev versus (reg & 1) << dev

In C, **<< has higher precedence than &**. So `reg & 1 << dev` means `reg & (1 << dev)`: it tests bit *dev*, which is what is wanted. In `(reg & 1) << dev` the brackets force the AND first: it extracts bit 0 only (the LIGHT bit) and then moves that result to position *dev*. It does not look at bit *dev* at all.

```
reg=00000100 (AC on): reg & 1 << 2 = 4 ; (reg & 1) << 2 = 0          r
eg=00000001 (LIGHT on, AC off): reg & 1 << 2 = 0 ; (reg & 1) << 2 = 4
```

With only AC on, the correct form gives 4 (non-zero, AC is on) but the wrong form gives 0 (AC looks off). With only LIGHT on, the wrong form gives 4 and falsely reports AC as on. A related trap: `reg & mask == 0` parses as `reg & (mask == 0)` because == binds tighter than &, so the test must be written `(reg & mask) != 0`.

What goes wrong with signed char when bit 7 is set

```
signed char : value=-128  s>>7=-1  (s>>7)==1:0  s==0x80:0  s<0:1  u
nsigned     : value=128  u>>7=1  (u>>7)==1:1  u==0x80:1  u<0:0   b
it test (s & (1<<7))!=0 -> 1 ; ~s as int = 127 ; ~u as int = -129(u
nsigned char)~u = 127
```

- Setting bit 7 makes a signed char **negative** (observed value -128, and `s < 0` is true). Converting 128 into signed char is implementation-defined.
- When a char is used in an expression it is promoted to int with **sign extension**. The register compares as -128, so `s == 0x80` is false (0x80 is 128) while the unsigned version gives true.
- Right shift of a negative value is implementation-defined; GCC does an **arithmetic shift**, filling with 1s, so `s >> 7` is -1, not 1, and `(s >> 7) == 1` is false.
- Printing the register as a number shows -128 instead of 128. The mask test `(s & (1 << 7)) != 0` still works, which hides the problem until another operation is used.
- Even for unsigned char, integer promotion applies: `~u` is the int -129, not an 8-bit value. The functions return unsigned char, so the result is truncated back to 8 bits, but intermediate results printed or compared must be cast: `(unsigned char)~u = 127`. **Rule: keep the register unsigned (unsigned char or uint8_t) and use 1u masks.**

3.5 Alternative concept: char state[8] (one byte per device)

```
void on(char s[8], int d)     { s[d] = 1; }
void off(char s[8], int d)    { s[d] = 0; }
void toggle(char s[8], int d) { s[d] ^= 1; }
int  is_on(char s[8], int d)  { return s[d]; }
int  count(char s[8])         { int n = 0; for (int i = 0; i < 8; i++) n += s[i]; return n; }
```

```
sizeof(unsigned char)=1 byte, sizeof(HomeArr)=8 bytes
1,000,000 homes: bit-register = 1000000 bytes, array = 8000000 bytes
count devices ON, 1,000,000 homes x 20 repeats (totals must agree: 79939780 79939780 79939780)
array scan     : 160000000 loop iterations per pass-set, 0.085 s
bit (Kernighan): 79939780 loop iterations per pass-set, 0.198 s b
it + 256-table: 20000000 lookups per pass-set,            0.008 s
```

Measure	One bit per device (unsigned char)	Array char state[8]
Time: set, clear, toggle, test	O(1): one shift plus one bitwise operation	O(1): one indexed access
Space per home	1 byte	8 bytes (sizeof = 8, measured)
Space for 1,000,000 homes	1,000,000 bytes (about 1.0 MB)	8,000,000 bytes (about 8.0 MB); 7 MB more
Count devices ON, per home	Kernighan loop: one iteration per ON bit; average 4.00 over all 256 register values (measured total 1024); a 256-entry table gives 1 lookup	always 8 iterations (2048 over all 256 states)
Count, 1,000,000 homes x 20 passes	79,939,780 iterations, 0.198 s (table version: 20,000,000 lookups, 0.008 s)	160,000,000 iterations, 0.085 s
Totals agree?	yes (79,939,780)	yes (79,939,780)

Analysis. Per-device operations are O(1) in both designs, and counting the ON devices of one home is O(8) = O(1) in both; over H homes counting is O(H). The bit register uses one eighth of the memory, which matters on a controller with a few bytes of RAM and when a million states are stored or sent over a network. The array is easier to read and extends beyond 8 devices without changing the type. Counting is the interesting case: the Kernighan loop does half the iterations of the array scan (4.0 on average versus 8) but was *slower* in wall-clock time here (0.198 s versus 0.085 s at -O2), probably because its loop length depends

on the data and the branch is hard to predict, whereas the array loop has a fixed length. The 256-entry lookup table (256 extra bytes) was fastest by a wide margin (0.008 s). Timings are machine-specific; the iteration counts are exact. The bit register also allows whole-register operations in one instruction (switch everything off with reg = 0, compare two states with ^).

Source code

```
#include <stdio.h>
#include <assert.h>
/* Q3: register trace with predictions, popcount, precedence and signedness experiments */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4, DEV7 = 7 };
unsigned char switch_on(unsigned char r, int d)  { return r | (1u << d); }
unsigned char switch_off(unsigned char r, int d) { return r & ~(1u << d); }
unsigned char toggle(unsigned char r, int d)     { return r ^ (1u << d); }
int is_on(unsigned char r, int d)                { return (r & (1u << d)) != 0; }
void bin(unsigned char r) { for (int b = 7; b >= 0; b--) putchar(((r >> b) & 1u) ? '1' : '0'); }

int count_on(unsigned char r, int *iters)   /* Kernighan: each pass clears the lowest set bit */
{
    int n = 0; *iters = 0;
    while (r) { r &= r - 1; n++; (*iters)++; }
    return n;
}

int main(void)
{
    printf("== Part 1: predicted vs observed register after each statement ==\n");
    struct { const char *stmt; int op, dev; unsigned char predicted; } steps[] = {
        {"switch_on(HEATER)",  0, HEATER, 0x08}, {"switch_on(PUMP)",     0, PUMP,   0x18},
        {"switch_on(FAN)",     0, FAN,    0x1A}, {"toggle(HEATER)",      2, HEATER, 0x12},
        {"toggle(LIGHT)",      2, LIGHT,  0x13}, {"switch_off(PUMP)",    1, PUMP,   0x03},
        {"switch_off(AC)",     1, AC,     0x03}, {"switch_on(device 7)",0, DEV7,   0x83},
    };
    unsigned char reg = 0;
    printf("%-22s %-9s %-4s %-9s %-4s %s\n", "statement", "pred.bin", "dec", "obs.bin", "dec", "match");
    for (int i = 0; i < 8; i++) {
        unsigned char before = reg;
        reg = steps[i].op == 0 ? switch_on(reg, steps[i].dev)
            : steps[i].op == 1 ? switch_off(reg, steps[i].dev) : toggle(reg, steps[i].dev);
        (void)before;
        printf("%-22s ", steps[i].stmt); bin(steps[i].predicted); printf(" %-4d ", steps[i].predicted);
        bin(reg); printf(" %-4d %s\n", reg, reg == steps[i].predicted ? "yes" : "NO");
    }
    int it; int n = count_on(reg, &it);
    printf("devices ON in final register = %d (loop iterations = %d)\n", n, it);

    printf("\n== Part 2: reg & 1 << dev  versus  (reg & 1) << dev ==\n");
    unsigned char t1 = 0x04, t2 = 0x01; int dev = 2;
    printf("reg=00000100 (AC on): reg & 1 << 2 = %d ; (reg & 1) << 2 = %d\n", t1 & 1 << dev, (t1 & 1) << dev);
    printf("reg=00000001 (LIGHT on, AC off): reg & 1 << 2 = %d ; (reg & 1) << 2 = %d\n", t2 & 1 << dev, (t2 & 1) << dev);

    printf("\n== Part 3: signed char with bit 7 set ==\n");
    signed char s = 0; s = s | (1 << 7);
    unsigned char u = 0; u = u | (1 << 7);
    printf("signed char : value=%d  s>>7=%d  (s>>7)==1:%d  s==0x80:%d  s<0:%d\n", s, s >> 7, (s >> 7) == 1, s == 0x80, s < 0);p
    rintf("unsigned     : value=%d  u>>7=%d  (u>>7)==1:%d  u==0x80:%d  u<0:%d\n", u, u >> 7, (u >> 7) == 1, u == 0x80, u < 0);p
    rintf("bit test (s & (1<<7))!=0 -> %d ; ~s as int = %d ; ~u as int = %d\n", (s & (1 << 7)) != 0, ~s, ~u);                pri
    ntf("(unsigned char)~u = %d\n", (unsigned char)~u);

    printf("\n== Part 4: count-ON cost over all 256 register values ==\n");
    long tot_k = 0;
    for (int v = 0; v < 256; v++) { int i2; count_on((unsigned char)v, &i2); tot_k += i2; }
    printf("Kernighan total iterations = %ld (average %.2f); array scan = 8 per register (%d total)\n", tot_k, tot_k / 256.0, 256 * 8);
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
/* Q3 alternative: one byte per device (char state[8]) versus one bit per device (unsigned char) */
#define HOMES 1000000
typedef struct { char state[8]; } HomeArr;

int main(void)
{
    printf("sizeof(unsigned char)=%zu byte, sizeof(HomeArr)=%zu bytes\n", sizeof(unsigned char), sizeof(HomeArr));
    printf("1,000,000 homes: bit-register = %zu bytes, array = %zu bytes\n",
           sizeof(unsigned char) * (size_t)HOMES, sizeof(HomeArr) * (size_t)HOMES);

    unsigned char *regs = malloc(HOMES);
    HomeArr *arr = malloc(sizeof(HomeArr) * HOMES);
    unsigned int s = 2463534242u;
    for (int i = 0; i < HOMES; i++) {
        s ^= s << 13; s ^= s >> 17; s ^= s << 5;
        regs[i] = (unsigned char)s;
        for (int d = 0; d < 8; d++) arr[i].state[d] = (regs[i] >> d) & 1;
    }
    unsigned char popc[256];
    for (int v = 0; v < 256; v++) { int c = 0; for (int b = 0; b < 8; b++) c += (v >> b) & 1; popc[v] = (unsigned char)c; }

    long total_a = 0, total_k = 0, total_t = 0, it_a = 0, it_k = 0;
    clock_t t0 = clock();
    for (int r = 0; r < 20; r++) for (int i = 0; i < HOMES; i++) { for (int d = 0; d < 8; d++) { total_a += arr[i].state[d]; it_a++; } }
```

```
    clock_t t1 = clock();
    for (int r = 0; r < 20; r++) for (int i = 0; i < HOMES; i++) { unsigned char v = regs[i]; while (v) { v &= v - 1; total_k++; it_k++; } }
    clock_t t2 = clock();
    for (int r = 0; r < 20; r++) for (int i = 0; i < HOMES; i++) total_t += popc[regs[i]];
    clock_t t3 = clock();
    printf("count devices ON, 1,000,000 homes x 20 repeats (totals must agree: %ld %ld %ld)\n", total_a, total_k, total_t);
    printf("array scan     : %ld loop iterations per pass-set, %.3f s\n", it_a, (double)(t1 - t0) / CLOCKS_PER_SEC);
    printf("bit (Kernighan): %ld loop iterations per pass-set, %.3f s\n", it_k, (double)(t2 - t1) / CLOCKS_PER_SEC);
    printf("bit + 256-table: %d lookups per pass-set,           %.3f s\n", 20 * HOMES, (double)(t3 - t2) / CLOCKS_PER_SEC);
    free(regs); free(arr);
    return 0;
}
```

Q4 - else-if Ladder or switch?

University grade and scholarship classifier. 6 marks, Bloom K4 (Analyse). Includes Complete the Code.

4.1 Problem and implementations

Grade rules: 90-100 S, 80-89 A, 70-79 B, 60-69 C, 50-59 D, below 50 F; marks outside 0-100 give '?'. Three implementations were written: an else-if ladder, a switch on `marks / 10` (TODO 1), and the lookup table from the alternative concept.

else-if ladder (student's program, Figure 4)

```
char grade(int marks) {
    if (marks < 0 || marks > 100) return '?';
    else if (marks >= 90) return 'S';
    else if (marks >= 80) return 'A';
    else if (marks >= 70) return 'B';
    else if (marks >= 60) return 'C';
    else if (marks >= 50) return 'D';
    else return 'F';
}
```

switch on marks / 10 (TODO 1 completed)

```
#include <stdio.h>
/* Grade rules: 90-100 S, 80-89 A, 70-79 B, 60-69 C, 50-59 D, below 50 F, invalid '?' */
char grade(int marks)
{
    if (marks < 0 || marks > 100) return '?';
    switch (marks / 10) {              /* TODO 1 */
        case 10:
        case 9: return 'S';
        case 8: return 'A';
        case 7: return 'B';
        case 6: return 'C';
        case 5: return 'D';
        default: return 'F';
    }
}
int main(void)
{
    int marks[] = {100, 95, 83, 50, 49, -5};
    for (int i = 0; i < 6; i++) printf("%d:%c ", marks[i], grade(marks[i]));
    printf("\n");
    return 0;
}
```

Observed output (both versions)	Expected output given in the assignment	Match
100:S 95:S 83:A 50:D 49:F -5:?	100:S 95:S 83:A 50:D 49:F -5:?	Identical

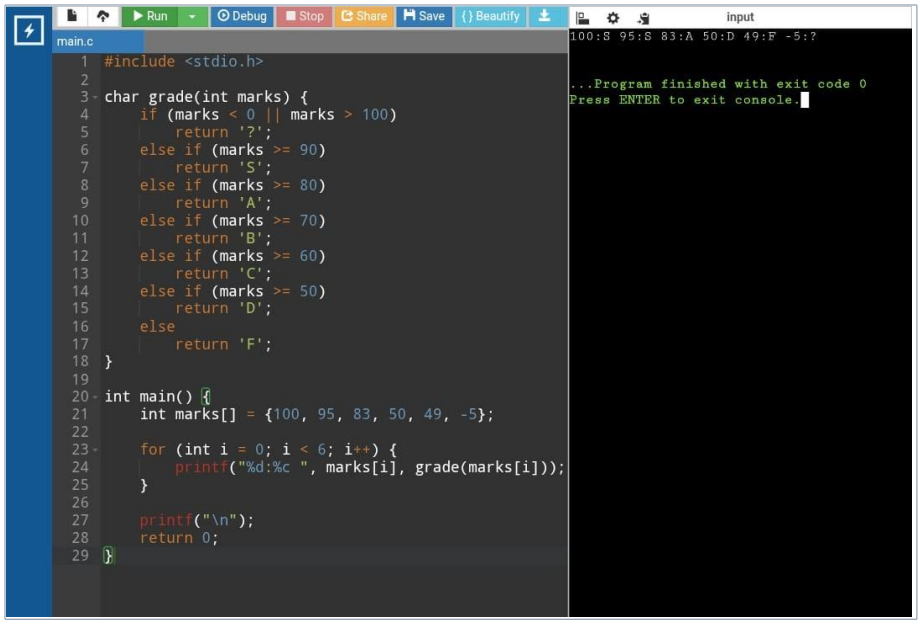


Figure 4 - Student's execution of the else-if ladder version.

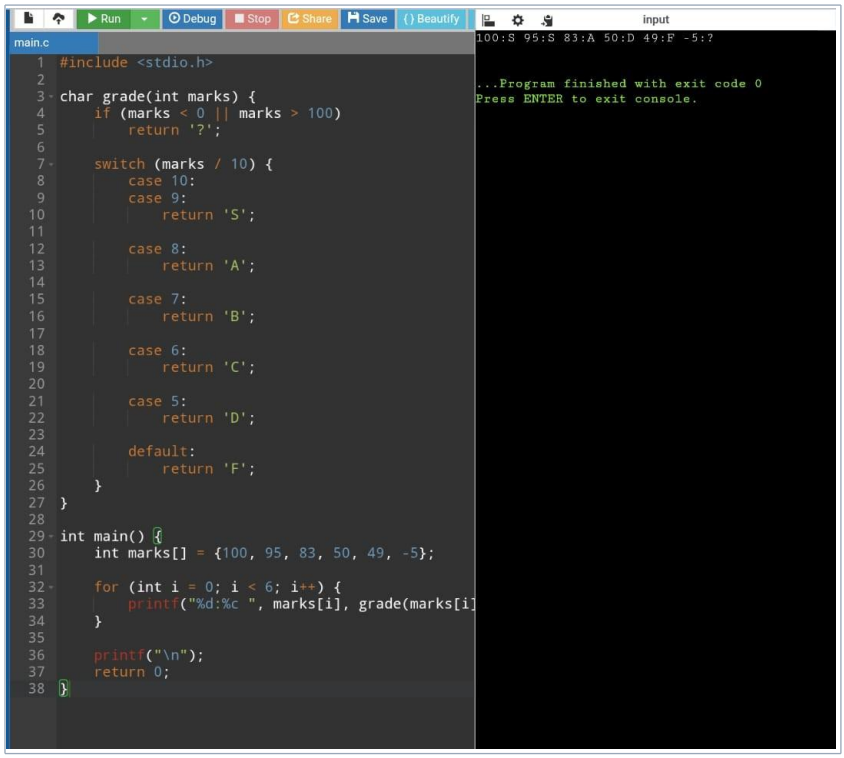


Figure 5 - Student's execution of the switch version (TODO 1 completed with case 10 and case 9 combined).

4.2 Which inputs are handled correctly and incorrectly

The six required inputs were tested on all three versions, with and without the validation line `if (marks < 0 || marks > 100) return '?';` (columns "no guard").

marks	Expected	Ladder	switch	Table	Ladder, no guard	switch, no guard	Table, no guard
100	S	S	S	S	S	S	S
90	S	S	S	S	S	S	S
89	A	A	A	A	A	A	A
0	F	F	F	F	F	F	F
-1	?	?	?	?	**F (wrong)**	**F (wrong)**	**F (wrong)**
101	?	?	?	?	**S (wrong)**	**S (wrong)**	**S (wrong)**

- With the guard all three versions are correct for all six inputs.
- Without the guard, -1 gives F and 101 gives S in **every** version. For the switch the reason is that integer division truncates toward zero, so -1 / 10 = 0 (case 0, default F) and 101 / 10 = 10 (case 10, S). The validation line is therefore essential, not optional. For the table, a value such as -10 or 110 would index outside the array (undefined behaviour), so the guard also protects memory.
- The switch is correct for 100 only because `case 10` is combined with `case 9`. If that label were forgotten, 100 / 10 = 10 would fall to the default and 100 would wrongly become F (by reasoning from the code).

4.3 Effect of a missing break (or return)

The switch was rewritten with `break` (program `q4_fallthrough.c`) so that one break could be removed at a time. A missing return behaves identically in the submitted version, because both end the case.

```
BUG=0: 100:S 95:S 83:A 75:B 65:C 50:D 49:F -5:?
BUG=1: 100:A 95:A 83:A 75:B 65:C 50:D 49:F -5:?
BUG=2: 100:S 95:S 83:B 75:B 65:C 50:D 49:F -5:?
BUG=3: 100:S 95:S 83:A 75:B 65:C 50:F 49:F -5:?
```

Variant	What was removed	Changed output	Why
BUG=0	nothing	none (reference)	-
BUG=1	break after case 9	100:A and 95:A (should be S)	Execution falls from case 9 into case 8 and overwrites S with A
BUG=2	break after case 8	83:B (should be A)	Falls from case 8 into case 7 and the grade becomes B
BUG=3	break after case 5	50:F (should be D)	Falls from case 5 into default and the grade becomes F

A missing break does not cause an error: the program compiles and runs, and the damage depends on the input. Only grades whose case precedes another case are affected, so a small test set can miss it. GCC reports it with `-Wextra` (-Wimplicit-fallthrough): "this statement may fall through". The deliberate empty `case 10: case 9:` is not flagged because it has no statements.

4.4 Why switch cannot directly test a range such as 75-84

- A case label must be an **integer constant expression** and is compared by **equality**. Relational tests such as `marks >= 75 && marks <= 84` are not allowed in a label.
- A range would need ten labels (case 75: ... case 84:), or the GCC extension `case 75 ... 84:`, which is not standard C.
- The expression `marks / 10` collapses 70-79 and 80-89 into buckets 7 and 8, so 75-84 straddles two buckets and cannot be expressed. Using `marks / 5` (75-79 gives 15, 80-84 gives 16) would work only because 75 and 85 are multiples of 5, and every other label would have to be rewritten.

4.5 Which version is easier to change when the grade boundaries change?

Version	Cost of changing a boundary (for example S from 90 to 85)
else-if ladder	Change one constant (90 to 85). Order of the tests must stay descending. Easiest.
switch on marks / 10	Boundaries must be multiples of the divisor. 85 is not, so the divisor becomes 5 and all case labels are rewritten.
lookup table	Boundary on a multiple of 10: edit one entry. Otherwise a larger table (up to 101 entries) or a different divisor. A data-driven variant (arrays of thresholds and letters scanned by a loop) is a one-line change.

4.6 Test set that exposes every boundary error

Boundary errors typically come from writing `>` for `>=` (or the reverse). To check the test set objectively, seven faulty versions (mutants) of the ladder were created: five with `>` at one grade threshold, and two with an off-by-one in the validation. The test set is {-1, 0, 49, 50, 59, 60, 69, 70, 79, 80, 89, 90, 99, 100, 101}.

```
mutant >90 (not >=)     killed by: 90   (1 tests)
mutant >80              killed by: 80   (1 tests)
mutant >70              killed by: 70   (1 tests)
mutant >60              killed by: 60   (1 tests)
mutant >50              killed by: 50   (1 tests)
mutant m<=0 invalid     killed by: 0   (1 tests)
mutant m>=100 invalid   killed by: 100   (1 tests)
```

Finding: every faulty version is exposed by the *exact* boundary value and by no other test. The neighbours (89, 99, 59, ...) do not detect a `>=` written as `>`. A test set built only from "typical" values such as 95, 83, 50 would detect none of the ladder errors except at 50. All 15 values passed on the three correct versions (failures = 0), and the three versions agreed on every integer from -20 to 120.

4.7 Lookup-table alternative and complexity

```
char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'};
char grade(int marks) {
    if (marks < 0 || marks > 100) return '?';
    return grades[marks / 10];
}
```

```
marks  100 -> 3 comparisons
marks   90 -> 3 comparisons
marks   89 -> 4 comparisons
marks    0 -> 7 comparisons
marks   -1 -> 1 comparisons
marks  101 -> 2 comparisons
all valid marks 0..100: total=603, average=5.97 comparisons
```

```
worst case = 7 comparisons (grade F)
three implementations agree on -20..120: yes
```

Measure	else-if ladder	switch (marks / 10)	Lookup table
Comparisons per grade	up to k: 3 for S, 7 for F (including the 2 validation tests); average 5.97 over 0-100	1 division, then a jump (the compiler chooses a jump table or a comparison tree)	1 division and 1 array read, plus the 2 validation tests
Time per grade	O(k) with k = 6 grade levels (a constant, so O(1) for fixed rules)	O(1)	O(1)
Space	none	none (code only)	11 bytes
Measured, 30,000,000 calls, -O0	0.145 s	0.120 s	0.099 s

The timing gap is small (30 million calls differ by less than 0.05 s) and machine-specific; it should not drive the decision. The comparison counts show the real difference: the ladder does more work for low grades, the table does a constant amount of work. For six fixed grades the ladder is perfectly adequate, and it is the only one of the three that handles arbitrary boundaries naturally.

4.8 Design decision: the scholarship rule

The assignment does not give the scholarship rule, so an **illustrative rule** was assumed: three income bands (low below Rs 3 lakh, middle Rs 3 to 6 lakh, high above Rs 6 lakh) and four categories. Replace the table with the real rule if it differs; the argument does not change.

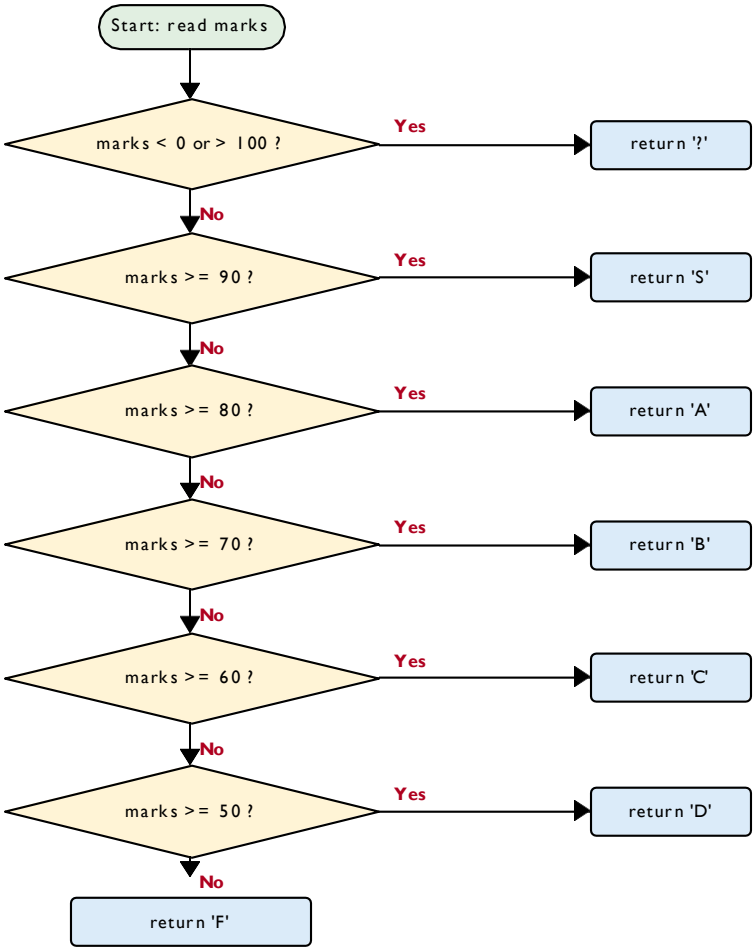
Grade	Low income	Middle income	High income
S	Full	Full	Half
A	Full	Half	Quarter
B	Half	Quarter	None
C	Quarter	None	None
D, F	None	None	None
? (invalid marks)	Rejected	Rejected	Rejected

Structure	Assessment
Nested if	Works, but the 6 x 3 rule is hidden in control flow: many branches, easy to mis-nest, every new grade or band edits several places, and the rule cannot be reviewed as a table.
switch inside an if	Moves the nesting into another statement. It reads better for the grade, but the income band still needs a second decision, and each grade case still repeats an inner if.
Lookup table (chosen)	The rule is data, not logic. One row per grade, one column per band; a reviewer can compare it directly to the policy. A new band is a new column. Lookup is O(1). Cost: a small constant table (18 entries) and an index for the grade.

Decision: a two-dimensional lookup table for grade x income band, with an if for the validation of marks and a short ladder to turn income into a band. The table cannot index income directly (income is a range, the same limitation as in 4.4), so the band is found by comparison, and the table is indexed by the band and the grade. In the experiment the table version and a fully nested-if version were compared on **42 cases** (7 grade values x 6 incomes, including the boundary incomes 29/30 and 60/61 and the invalid grade): **0 differences**.

```
cells tested=42, differences=0
grade S: low=Full     mid=Full     high=Half
grade A: low=Full     mid=Half     high=Quarter
grade B: low=Half     mid=Quarter  high=None
grade C: low=Quarter  mid=None     high=None
grade D: low=None     mid=None     high=None
grade F: low=None     mid=None     high=None
```

Flowchart of the else-if ladder



Pseudocode

```
FUNCTION grade(marks)
    IF marks < 0 OR marks > 100 THEN RETURN '?'
    index <- marks DIV 10                  (10 for 100, 9 for 90-99, ..., 0 for 0-9)
    RETURN GRADES[index]                   (table: F F F F F D C B A S S)

FUNCTION scholarship(grade, income)
    IF grade = '?' THEN RETURN "Rejected"
    band <- 0 IF income < 3 lakh, 1 IF income <= 6 lakh, ELSE 2
    RETURN TABLE[row of grade][band]
```

Key source excerpt (full source in Q4/q4_tests.c)

```
static const char *T[6][3] = {
    /*        low         mid         high  */
    /* S */ {"Full",    "Full",    "Half"},
    /* A */ {"Full",    "Half",    "Quarter"},
    /* B */ {"Half",    "Quarter", "None"},
    /* C */ {"Quarter", "None",    "None"},
    /* D */ {"None",    "None",    "None"},
    /* F */ {"None",    "None",    "None"} };
const char *order = "SABCDF"; const char *p = strchr(order, g);
if (!p) return "Rejected";
int band = income < 30 ? 0 : income <= 60 ? 1 : 2;   /* income in units of Rs 10,000 */
return T[p - order][band];
```

Summary of Results (Q1 to Q4)

Q	Main result	Key evidence
Q1	The one-line expression charges GST on the gross amount and truncates in integer division; count = count++ + 1 is undefined behaviour.	Bill A: 1080 versus intended 1071 (+Rs 9); Bill B: 267 versus 265.07; -Wall gives -Wsequence-point. Intermediate variables: 8 operations instead of 10.
Q2	Guarding the slow call with the cheap card check reduces slow calls from 1000 to 78-89 per 1000 passengers.	V1 = 78, V2 = 89, V3 = V4 = 1000, nested if = 78. With an audit log, card-first order (V2) is required if every card holder must be logged.
Q3	One byte holds 8 devices; | sets, & ~ clears, ^ toggles, & tests. Output matches the expected output exactly.	8 of 8 predicted registers matched; reg & 1 << dev = 4 versus (reg & 1) << dev = 0; 1 MB versus 8 MB for a million homes.
Q4	Ladder and switch give identical grades for valid and invalid marks only when the range check is present; a missing break changes results silently.	15-value boundary set kills 7 of 7 mutants; each is caught only by the exact boundary value; ladder average 5.97 comparisons versus 1 table read.

Appendix A - Repository layout and build commands

Folder names follow the assignment (Q1 to Q4). Each folder holds the source, the captured text output and the screenshots. At least one commit per question is required.

```
CSA02-<RegisterNumber>-Assignment2/
  README.md                    (name, register no., GCC version, commands, short result notes)
  Q1/  q1_compare.c  q1_steps.c  q1_count.c  + output screenshots
  Q2/  q2_experiment.c  + output screenshots (single-passenger program and the 1000-passenger experiment)
  Q3/  q3_complete.c  q3_trace.c  q3_array.c  + output screenshots
  Q4/  q4_switch.c  q4_fallthrough.c  q4_tests.c  q4_bench.c  + output screenshots
```

Program	Compile	Run
Q1	gcc -Wall -Wextra -o q1_compare q1_compare.c	./q1_compare
Q1 (UB warning)	gcc -Wall -Wextra -o q1_count q1_count.c	./q1_count
Q2	gcc -Wall -Wextra -O0 -o q2_experiment q2_experiment.c	./q2_experiment
Q3	gcc -Wall -Wextra -o q3_complete q3_complete.c	./q3_complete
Q3 (trace)	gcc -Wall -Wextra -o q3_trace q3_trace.c	./q3_trace
Q3 (array)	gcc -Wall -Wextra -O2 -o q3_array q3_array.c	./q3_array
Q4	gcc -Wall -Wextra -o q4_switch q4_switch.c	./q4_switch
Q4 (break test)	gcc -DBUG=1 -o q4_fallthrough q4_fallthrough.c (BUG = 0, 1, 2 or 3)	./q4_fallthrough
Q4 (tests)	gcc -Wall -Wextra -o q4_tests q4_tests.c	./q4_tests
Q4 (timing)	gcc -O0 -o q4_bench q4_bench.c	./q4_bench

Compiler for all measured data: GCC 13.3.0 on Ubuntu 24.04. The screenshots (Figures 1 to 5) come from the student's own runs; every other count in this report was produced by the programs listed above.